

P1091R3

Extending structured bindings to be more like variable declarations

Published Proposal, 2019-02-18

This version:

<https://wg21.link/p1091r3>

Author:

Nicolas Lesser (blitzrakete[at]gmail[dot][com])

Audience:

CWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

Making structured bindings more like first-class-citizen.

Table of Contents

1	This revision
2	Changes
3	Introduction
4	Motivation
5	Linkage
6	Extern
7	Static and thread_local
8	Inline
9	Constexpr
10	Lambda captures
11	maybe-unused
12	Template
13	Impact
14	Proposed wording
15	Acknowledgements

§ 1. This revision

This revision contains only wording for the changes approved by EWG:

- By value capture of structured bindings
- static and thread_local for structured binding declarations

and various proposed wording fixes by Core.

§ 2. Changes

r0:

- Initial revision

r1:

- Better wording for Core
- Lambda captures
- `[[maybe_unused]]`
- SB templates

r2:

- Just wording

r3:

Wording as reviewed by Core for what EWG approved:

- by-value capture of SBs
- `static` and `thread_local`
- original motivation

§ 3. Introduction

There are a lot of restriction on structured bindings compared to variable declarations, like not being able to mark them `static`, `constexpr` or them not having unclear linkage. This proposal's aim is to fix this by for example making the underlying structured binding object (and tuple binding variables) have external linkage and by allowing various specifiers (`static`, `thread_local`, `constexpr`, `inline`, `extern`) on structured bindings.

§ 4. Motivation

Structured bindings, although very useful, are actually pretty magical. They don't introduce variables in the normal sense for each binding, rather, they are names that refer to specific objects. As such, there are problems including what it means for a binding to be `static` and how it would work and what linkage do those bindings even have.

Those problems could not be resolved during the discussion of the paper and afterwards, a paper was requested to analyse the possible design impact that such additions to structured bindings would have after two NB comments proposing this were rejected. This paper attempts to do so.

One motivation to do so is to bring structured bindings closer to actual variable declarations, so consistency. This will also make structured bindings more useful, as they are currently lacking for example `constexpr`, which is becoming every more important for various features of the language.

As a consequence this paper also fixes some DRs that were filed and are under consideration or going to be eventually by either Evolution or Core.

§ 5. Linkage

As per [basic.link]p8, bindings do not have any linkage because they're just names. Currently, in both gcc and clang, tuple binding have linkage, as they were specified as variable declarations in the standard. This is no longer the case though due to the resolution of DR2313¹.

But the underlying structured binding object is an actual variable, which can have either internal or external linkage depending on the declaration of the structured binding, as gcc and clang do it. There is no way to refer to that object though without making the program IF-NDR as the object has a name like `_ZDC1a2bbE` for `auto[a, bb]`.

[dcl.struct.bind]p1 has the following to say about the object:

First, a variable with a unique name *e* is introduced.

It follows that the variable cannot be referenced in a conforming program anyways, and as such, it doesn't make much sense to give it external linkage. Nonetheless, to be consistent with the rest of the declarations and being able to use just `inline` (see below) without an extra `extern` to give the structured binding external linkage, the underlying object should have external linkage.

§ 6. Extern

As discussed in the previous section, there is no way to reference either the bindings or the underlying object within a conforming program, so allowing `extern` on such a structured binding would not make much sense.

However, this would prohibit `inline` on structured bindings that have been declared `const` and `inline`, which might be desirable in some cases. For this reason, it should be allowed. Note that `extern` would have no effect on the individual bindings, except for the tuple case.

§ 7. Static and `thread_local`

`static` and `thread_local` on a structured binding make sense and are actually useful. The way to make this work nicely in the standard is to only apply them on the underlying object, and not on the bindings (which wouldn't make sense and can't work today without major changes to the specification of bindings anyways). For the tuple case on the additional variable declarations too.

Because bindings refer to certain objects (depending on the initializer of the structured binding), they would implicitly get the desired semantics of `static` and `thread_local`, as they refer to either objects within the underlying object or to separate variables (in the tuple case) which are marked with the desired specifiers.

§ 8. Inline

`inline` is also useful and will be consistent with inline variables. It will work just as with `static` and `thread_local`: The underlying object is marked `inline` and any additional variables introduced as part of tuple bindings.

§ 9. `constexpr`

If `constexpr` were applied just like `static` and co. are, then there would be a problem, because the current language rules make the following code ill-formed:

```
// at block scope
constexpr auto[a] = std::tuple<int>(1);

// "equivalent" to

constexpr auto __sb = std::tuple<int>(1);

constexpr const int& __a = std::get<0>(__sb); // ill-formed today
```

A reference must be initialized by a constant expression to be a core constant expression ([expr.const]p2.11], but `std::get<0>(__sb)` is not a constant expression due to [expr.const]p6.

Richard Smith on the core reflector² suggested to relax the restriction on what constitutes a core constant expression of a reference by just requiring that the reference must be initialized by a *core* constant expression instead (see below to what this change entails).

This would mean that to make structured bindings `constexpr`, it is necessary to apply `constexpr` to the underlying object, and apply `const` to any other variable introduced by tuple bindings.

Of course, one thing to note is that it is important to guarantee that the call to `get` is a constant expression, because or else `constexpr` will act like `const`, which is only maybe a core constant expression.

§ 10. Lambda captures

Lambda captures aren't currently allowed to refer to a structured binding. There seems to be no technical reason to disallow this, and indeed, the wording for allowing this just removes the restriction on capturing structured bindings.

§ 11. maybe-unused

Currently, `[[maybe_unused]]` cannot be applied to a structured binding declaration. There doesn't seem to be a good reason to disallow this; EDG, clang and gcc all already support `[[maybe_unused]]` on structured bindings.

§ 12. Template

The following code is ill-formed as of C++20:

```
template <auto Var>constexpr auto[X, Y] = Var;
```

As it uses 1) a structured binding declaration template and 2) it is constexpr. The latter is already handled. Should a structured binding be a valid *template-declaration*? The author argues that yes, it should be. It allows for code that can decompose any non-type template parameter (which now can be any class type, thanks to P0732r2).

```
constexpr std::pair Position(1, 2);constexpr std::pair Flag(4, 5);  
  
if (X<Position> == Y<Flag>)  
    ; // almost there!
```

Note: The author doesn't know how to change the standard to allow this completely.

§ 13. Impact

This proposal only makes ill-formed or code with unspecified behavior well-formed in relation to structured bindings.

Due to `constexpr` tuple binding variables requiring a change to what constitutes a core constant expression, ill-formed code today will become well-formed:

```
// at block scope  
const int var = 1;  
  
const int& ref = var;  
  
static_assert(ref == 1); // ill-formed today, well-formed with this prop
```

§ 14. Proposed wording

Change [expr.prim.lambda.capture]p8 (7.5.5.2) as follows:

If a *lambda-expression* explicitly captures an entity that is not odr-usable ~~or captures a structured binding (explicitly or implicitly)~~, the program is ill-formed.

Change [expr.prim.lambda.capture]p12 (7.5.5.2) as follows:

A bit-field , a structured binding, or a member of an anonymous union shall not be captured by reference.

Change [dcl.struct.bind]p1 (9.5) as follows:

Let *cv* denote the *cv-qualifiers* in the *decl-specifier-seq* and *S* consist of the storage-class-specifiers of the decl-specifier-seq (if any). [...] If the *assignment-expression* in the *initializer* has array type *A* and no *ref-qualifier* is present, *e* ~~has type cv *A*~~ is defined by

*attribute-specifier-seq*_{opt} *S* cv *A* *e*;

and each element is copy-initialized or direct-initialized from the corresponding element of the *assignment-expression* as specified by the form of the *initializer*.

Change [dcl.struct.bind]p4 (9.5) as follows:

Given the type *T_i* designated by `std::tuple_element<i, E>::type` and the type *U_i* designated by either *T_i&* or *T_i&&*, where *U_i* is an lvalue reference if the initializer is an lvalue and an rvalue reference otherwise, variables are introduced with unique names *r_i* as follows:

S *U_i* *r_i* = *initializer*;

~~of type "reference to *T_i*" initialized with the initializer, where the reference is an lvalue reference if the initializer is an lvalue and an rvalue reference otherwise.~~

Change [dcl.dcl]p8 (10) completely as follows:

~~If the decl-specifier-seq shall~~ contain s any decl-specifier other than static, thread local, only the type-specifier `auto`, ~~and~~ or *cv-qualifiers* , the program is ill-formed.

Change [dcl.stc]p3 (10.1.1) as follows:

The `thread_local` specifier indicates that the named entity has thread storage duration. It shall be applied only to the names of variables of namespace or block scope, to structured binding declarations ([dcl.struct.bind]), and to the names of static data members.

Change [dcl.stc]p4 (10.1.1) as follows:

The `static` specifier can be applied only to names of variables and functions, to structured binding declarations ([dcl.struct.bind]), and to anonymous unions.

§ 15. Acknowledgements

Thanks to Jens for reviewing the first draft of the proposed wording.